C05 ASSESSMENT TOOL 1(Constraint Problem Solving)

Name: Y.Thanushma

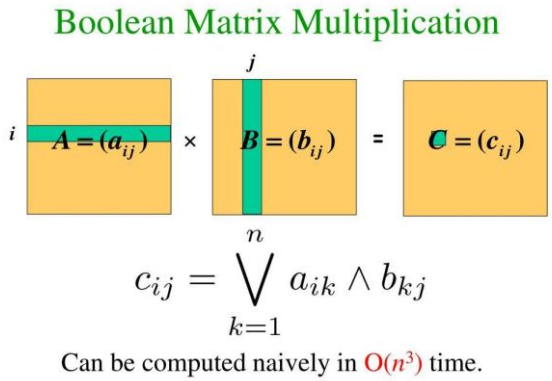
Registration Number: 192425355

Course Code: CSA0617-Design & Analysis of Algorithms

1. BOOLEAN MATRIX FACTORIZATION (NP-HARD PROBLEM)

Problem:

Given a Boolean matrix, factorize it into two smaller Boolean matrices under logical constraints. The objective is to reconstruct the original matrix using Boolean operations while minimizing the number of factors.



Constraints:

1. All matrix elements must be either 0 or 1.
2. Boolean multiplication uses AND (∧).
3. Boolean addition uses OR (∨).
4. Reconstruction must satisfy:

   Mij = ∨k (Aik ∧ Bkj)

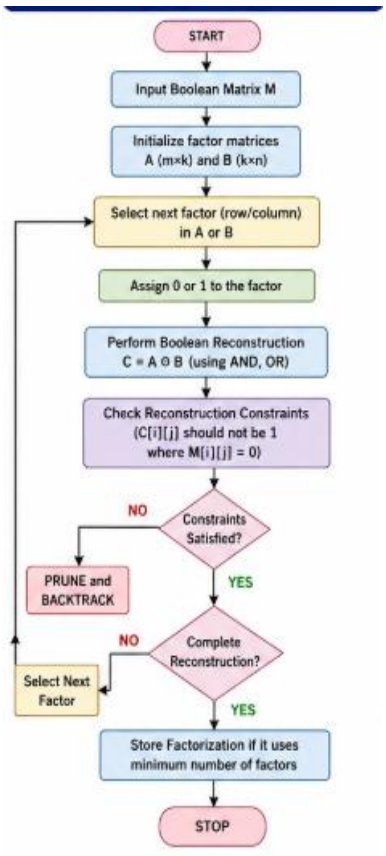
5. The number of factors k should be as small as possible.

Effect of Constraints:

The Boolean restrictions reduce the number of possible factor combinations. A factor is feasible only if it does not create a 1 in a position where the original matrix contains 0. Therefore, many combinations can be rejected early.

Backtracking Approach:

1. Start with an empty set of Boolean factors.
2. Select a possible row/column factor.
3. Assign 0 or 1 to the factor.
4. Reconstruct the corresponding part of the matrix.
5. Check whether any reconstructed 1 occurs where M has 0.
6. If inconsistent, backtrack immediately.
7. Continue until the complete matrix is reconstructed.
8. Keep the factorization using the minimum number of factors.



Pruning:

1. Reject a factor if it produces an invalid 1.
2. Stop exploring a branch if the current number of factors is already larger than the best solution.
3. Reject duplicate or equivalent factor combinations.
4. Check partial reconstruction before assigning remaining variables.

Correctness:

Every valid factor combination is explored unless it is pruned. Pruning removes only combinations that cannot produce a valid solution. Therefore, the final best factorization satisfies the Boolean reconstruction constraints and uses the minimum number of factors.

Complexity:

Boolean Matrix Factorization is NP-Hard because finding the minimum number of Boolean factors involves searching an exponentially large space of possible factorizations. Backtracking has exponential worst-case complexity, although pruning can significantly reduce practical search.

2. MINIMUM FEEDBACK EDGE SET PROBLEM (NP-COMPLETE PROBLEM)

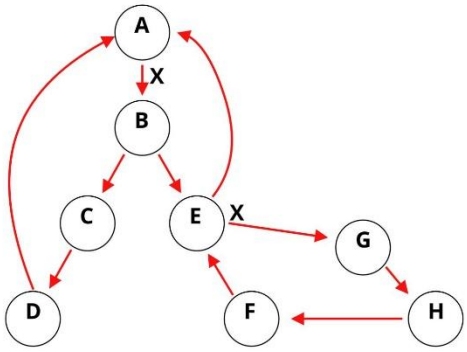
Problem:

Given a graph G = (V, E), find the smallest set of edges whose removal makes the graph acyclic.

Let F ⊆ E be the selected feedback edge set. The objective is:

$$\text{Minimize } |F|$$

such that: $G' = (V, E - F)$ contains no cycles.



Constraints:

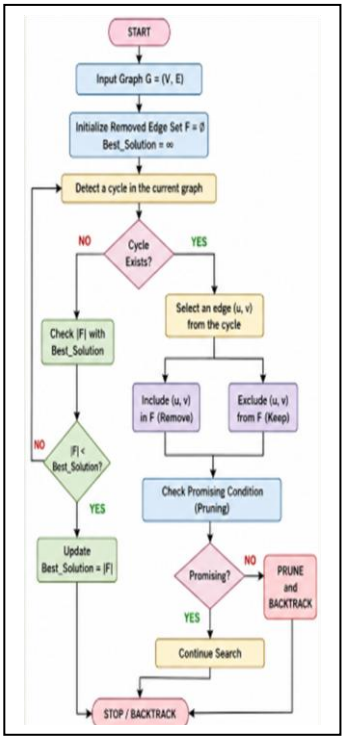
1. Only existing edges can be selected for removal.
2. Every cycle must contain at least one removed edge.
3. The remaining graph must be acyclic.
4. The number of removed edges should be minimum.

Effect of Constraints:

The constraints restrict feasible subsets because removing an arbitrary edge is not necessarily useful. An edge should preferably break one or more cycles. A candidate solution is valid only when all cycles are eliminated.

Backtracking Approach:

1. Start with an empty set of removed edges.
2. Detect a cycle in the current graph.
3. Select an edge from that cycle.
4. Branch into two possibilities:
    - Remove the edge.
    - Keep the edge.
5. After each decision, check whether the graph is still cyclic.
6. If no cycle remains, record the solution.
7. Backtrack and search for a smaller solution.



Pruning:

1. If the number of removed edges is already greater than or equal to the best known solution, stop.
2. If an edge does not belong to any remaining cycle, avoid considering it for removal.
3. If a cycle remains, focus only on edges belonging to that cycle.
4. Stop immediately when an acyclic graph is obtained with fewer removals.

Correctness:

The algorithm considers the necessary choices for breaking every detected cycle. A branch is accepted only when the resulting graph is acyclic. Since all relevant combinations are explored and larger solutions are pruned, the best recorded solution is a minimum feedback edge set.

Complexity:

The decision version—whether a feedback edge set of size at most k exists—is NP-Complete. The optimization problem is NP-Hard. Backtracking has exponential worst-case complexity because it may examine many subsets of edges.

3. CONSTRAINT-BASED FILE PLACEMENT (NP-HARD PROBLEM)

Problem:

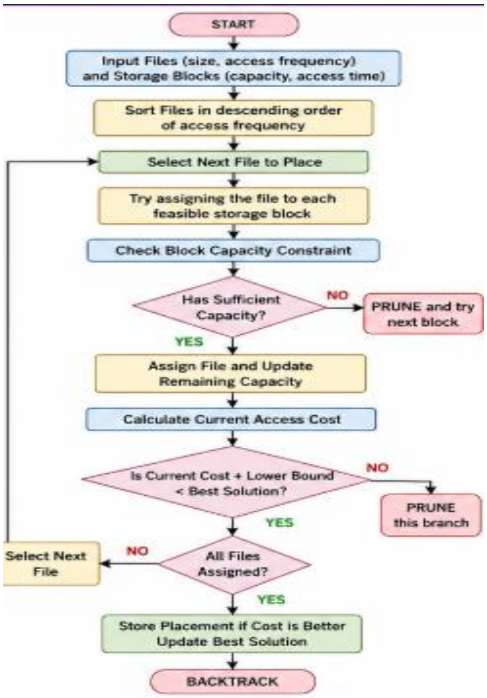
Assign files to available storage blocks so that the total access time is minimized while satisfying storage constraints.

The objective can be represented as: **Minimize Σi (fi × ti)**

where:

fi = access frequency of file i

ti = access time resulting from its assigned block.



Constraints:

1. Every file must be assigned to an available block.
2. A block cannot exceed its storage capacity.
3. A file cannot be split if splitting is not permitted.
4. Frequently accessed files should preferably be placed in faster or closer blocks.
5. Each block can contain only files whose total size fits its capacity.

Effect of Constraints:

Storage capacity limits the available placements. High-frequency files have a greater effect on total access time, so assigning them to slow blocks can produce an inefficient solution. Therefore, file size, access frequency, capacity, and access time must be considered together.

Backtracking Approach:

1. Sort files according to access frequency or importance.
2. Select the next file.
3. Try assigning it to each feasible storage block.
4. Update the remaining block capacity.
5. Calculate the current access cost.
6. Continue assigning the remaining files.
7. When all files are assigned, record the total cost.
8. Backtrack to search for a better placement.

Pruning:

1. Reject a placement if the block does not have enough capacity.
2. Calculate a lower bound on the remaining access cost.
3. Stop if the current cost plus the lower bound is already greater than the best solution.
4. Prefer faster blocks for highly accessed files.
5. Eliminate assignments that are equivalent or clearly dominated by better placements.

Correctness:

Every feasible file assignment is considered unless it is safely eliminated by pruning. Capacity constraints are checked before every assignment, so invalid placements are never accepted. Among all feasible assignments, the solution with the lowest total access cost is selected.

Complexity:

The problem can require exploring many possible file-to-block assignments. In the general constrained case, this produces an exponential search space, making the optimization problem NP-Hard. Backtracking with pruning can reduce the practical search space considerably, but the worst-case complexity remains exponential.